



Data Visualization

LABORATORIO

□□□□□□□□ □ □□□□□□□□□□□□



Argomenti del giorno

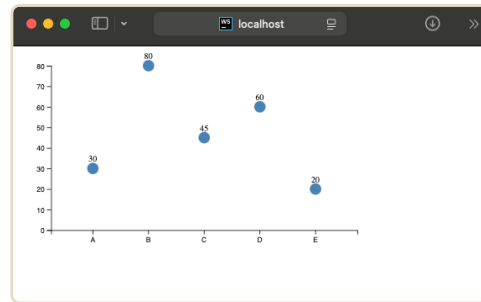
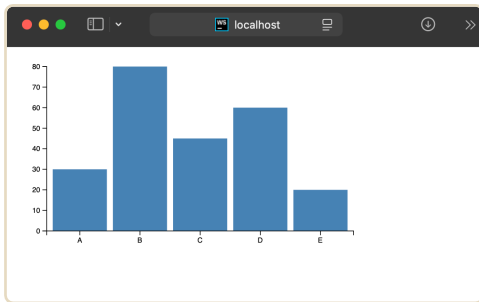
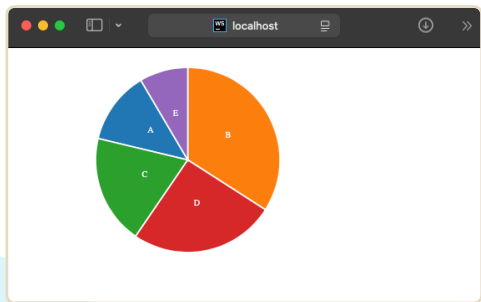
- **Scale Functions**
- **Line chart**

Cos'è una scale function

Le **scale functions** in D3.js sono funzioni che **mappano** un insieme di valori di **input** (dominio, i tuoi dati) a un insieme di valori di **output** (codominio, solitamente pixel).

Sono fondamentali nella creazione di **grafici** e visualizzazioni, perché permettono di **adattare** i dati numerici a coordinate, colori, dimensioni e altri attributi visivi in modo proporzionale e leggibile.

```
// 📊 Dati di esempio (categorie con valori numerici)
const data = [
  { categoria: "A", valore: 30 },
  { categoria: "B", valore: 80 },
  { categoria: "C", valore: 45 },
  { categoria: "D", valore: 60 },
  { categoria: "E", valore: 20 }
];
```

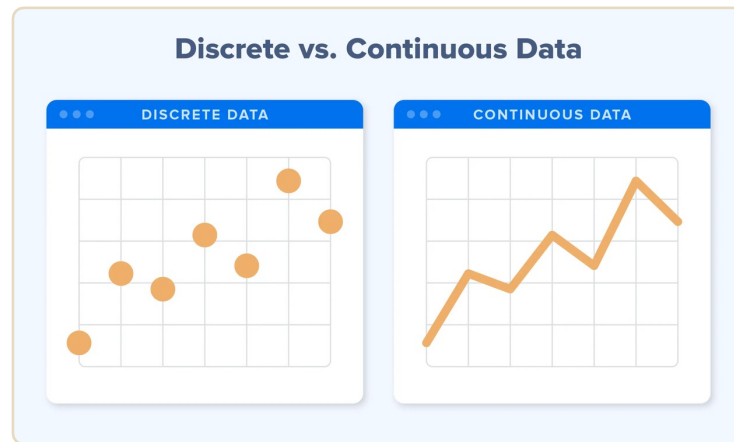


Cos'è una scale function

Senza **scale functions**, bisognerebbe calcolare **manualmente** ogni posizione o valore visivo, rendendo il codice **meno flessibile** e più difficile da mantenere.

Le **scale functions** si suddividono in quattro categorie principali, in base alla natura del dominio e del codominio.

DOMINIO		CODOMINIO
continuo	→	continuo
continuo	→	discreto
discreto	→	continuo
discreto	→	discreto



Esempi di scale functions

C2C

- **scaleLinear**
- scalePow
- scaleSqrt
- scaleLog10
- **scaleTime**
- scaleSequential

C2D

- scaleThreshold

D2C

- scaleBand
- scalePoint

D2D

- scaleOrdinal

Costruzione di un linechart

Costruiamo un linechart che ci indichi l'andamento del prezzo dei bitcoin dal 2013 al 2018

Per visualizzare i dati, li processiamo con la funzione **d3.csv()** e li salviamo in una costante.

La funzione csv di D3:

- legge il CSV da un URL o file locale
- usa la prima riga come nomi delle colonne
- restituisce una promise che crea un array di oggetti

Sia i valori di **date** e **value** sono delle **stringhe**, perciò dobbiamo rielaborarle come **Date** e **Number**.

```
async function getData() { Show usages
  const csv_path = "../dataset/bitcoin.csv";
  const data = await d3.csv(csv_path);
  console.log(data);
  return data
}
```

usiamo il codice base linechart...

```
▼ [O] result: Array (1822)
0 ▶ {date: "2013-04-28", value: "135.98"}
1 ▶ {date: "2013-04-29", value: "147.49"}
2 ▶ {date: "2013-04-30", value: "146.93"}
3 ▶ {date: "2013-05-01", value: "139.89"}
4 ▶ {date: "2013-05-02", value: "125.6"}
5 ▶ {date: "2013-05-03", value: "108.13"}
6 ▶ {date: "2013-05-04", value: "115"}
7 ▶ {date: "2013-05-05", value: "118.8"}
8 ▶ {date: "2013-05-06", value: "124.66"}
9 ▶ {date: "2013-05-07", value: "113.44"}
```

Preprocessing dei dati

Creiamo una funzione **preprocess data**:
La funzione **d3.timeParse()** permette di **convertire** una **stringa** in tipo **Date**, per farlo però bisogna definire il formato della data nella stringa, nel nostro caso 'YYYY-mm-DD'.

La funzione map ci permette di applicare a ogni oggetto s :

- **...s** copia tutte le proprietà dell'oggetto così come sono
- poi facciamo una conversione del valore del campo date con timeParser di D3
- Per il campo value, ci basta aggiungere un **+** davanti a una stringa per convertirla **implicitamente** in un numero

```
async function preprocessData(data) { Show usages
  const dataStringFormat = "%Y-%m-%d"
  const timeParser = d3.timeParse(dataStringFormat);
  return data.map(row => ({
    ...row,
    'date': timeParser(row['date']),
    'value': +row['value']
  }))
}
```

```
async function main() { Show usages
  let data = await getData();
  let processedData = await preprocessData(data);
  renderData(processedData);
}
```

Definizione dimensioni

Recuperiamo l'svg che racchiuderà il **grafico** e ci salviamo le sue **dimensioni**

Creiamo un oggetto **margin** che conterrà i valori di tutti i margini del nostro grafico.

Le dimensioni del **grafico** saranno date dalla dimensione dell'svg meno la i vari margini..

```
function renderData(data) { Show usages
  //dimensioni dell'SVG
  const svg = d3.select("svg");
  const wSvg = svg.attr("width");
  const hSvg = svg.attr("height");

  //margini del grafico
  const margin = {
    top: 40,
    right: 60,
    bottom: 40,
    left: 60
  };

  //dimensioni grafico
  const wChart = wSvg - margin.left - margin.right;
  const hChart = hSvg - margin.top - margin.bottom;
```


Aggiunta degli elementi

Dopo aver recuperato l'SVG, **aggiungiamo** un gruppo che conterrà il nostro **linechart**.

Siccome lo vogliamo **centrato**, lo trasliamo utilizzando i margini definiti prima.

```
//aggiungiamo un gruppo all'svg
const gContainer = svg
  .append('g')
  .attr("transform", //lo trasliamo per centrarlo
    `translate(${margin.left},${margin.top})`)
```

Scale functions nel grafico

Adesso i nostri dati sono formattati correttamente, è ora di usare la **scale functions** per **mappare** i valori e le date nel range che visualizzeremo nel grafico.

Tramite **d3.scaleTime()** mappiamo le date del dataset nell'asse x del nostro grafico, mentre tramite **d3.scaleLinear()** mappiamo invece i valori.

In questo caso le due scale function fanno la stessa cosa, ma una opera con valori numerici mentre l'altra con valori temporali.

```
const scaleX = d3.scaleTime()
  .domain(d3.extent(data, d => d.date))
  .range([0, wChart])
  .nice()

const scaleY = d3.scaleLinear()
  .domain(d3.extent(data, d => d.value))
  .range([hChart, 0])
  .nice()
```

- **.domain()** (intervallo dell'input): è una funzione della scala che definisce l'intervallo dei dati in ingresso
- **.range()** (intervallo dell'output): il range di pixel da usare per mappare i dati
- **.nice()** arrotonda i valori del dominio

Generazione del path

Per rappresentare i dati in un line chart è necessario utilizzare un **'path'**. Dato che andremo a crearlo **dinamicamente**, non è necessario specificare tutte le coordinate, ma passeremo i nostri dati alla funzione `.datum()`

`.datum()` differisce da `.data()` perchè permette di passare tutti i dati di una collezione a un **solo** elemento, mentre `.data()` crea un elemento per ogni dato.

Per riempire l'intera area del line chart è necessario utilizzare la funzione `d3.area()` quando si definiscono i valori dell'attributo **'d'**.

Bisogna quindi specificare le **coordinate** x, definite dalle **date**, la y iniziale che sarà **0** e la y finale che è definita dai **valori** della collezione.

Tramite gli attributi **fill** e **stroke** definiamo il colore.

```
gContainer
  .append("path")
  .datum(data)
  .attr("fill", "lightskyblue")
  .attr("stroke", "lightskyblue")
  .attr("d", d3.area()
    .x(d => scaleX(d.date))
    .y0(scaleY(0))
    .y1(d => scaleY(d.value)))
```

Grafico ad area

```
gContainer
  .append("path")
  .datum(data)
  .attr("fill", "lightskyblue")
  .attr("stroke", "lightskyblue")
  .attr("d", d3.area()
    .x(d => scaleX(d.date))
    .y0(scaleY(0))
    .y1(d => scaleY(d.value)))
```

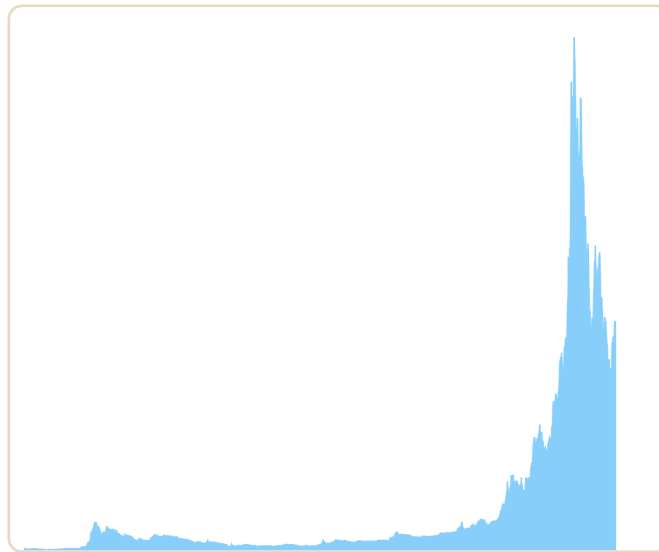


Grafico lineare

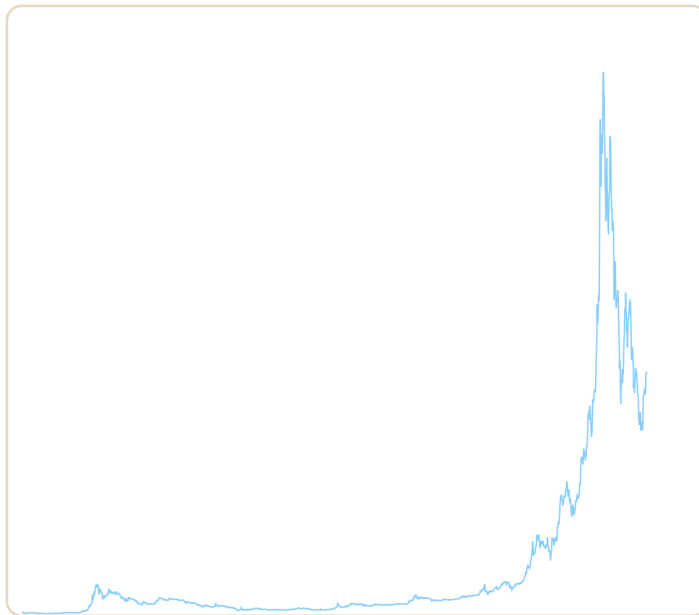
Se vogliamo invece definire solo il path senza riempire l'area, invece di usare `d3.area()` usiamo **`d3.line()`** che richiederà le **coordinate** x e y che seguono la stessa logica della funzione precedente.

Anche in questo caso è necessario definire **stroke** e **fill**, quest'ultimo perché se non impostato su **'none'** ci restituirà un comportamento inaspettato.

```
gContainer
  .append("path")
  .datum(data)
  .attr("fill", "none")
  .attr("stroke", "lightskyblue")
  .attr("d", d3.line()
    .x(d => scaleX(d.date))
    .y(d => scaleY(d.value)))
```

Grafico lineare

```
gContainer
  .append("path")
  .datum(data)
  .attr("fill", "none")
  .attr("stroke", "lightskyblue")
  .attr("d", d3.line()
    .x(d => scaleX(d.date))
    .y(d => scaleY(d.value))
  )
```



Aggiunta degli assi

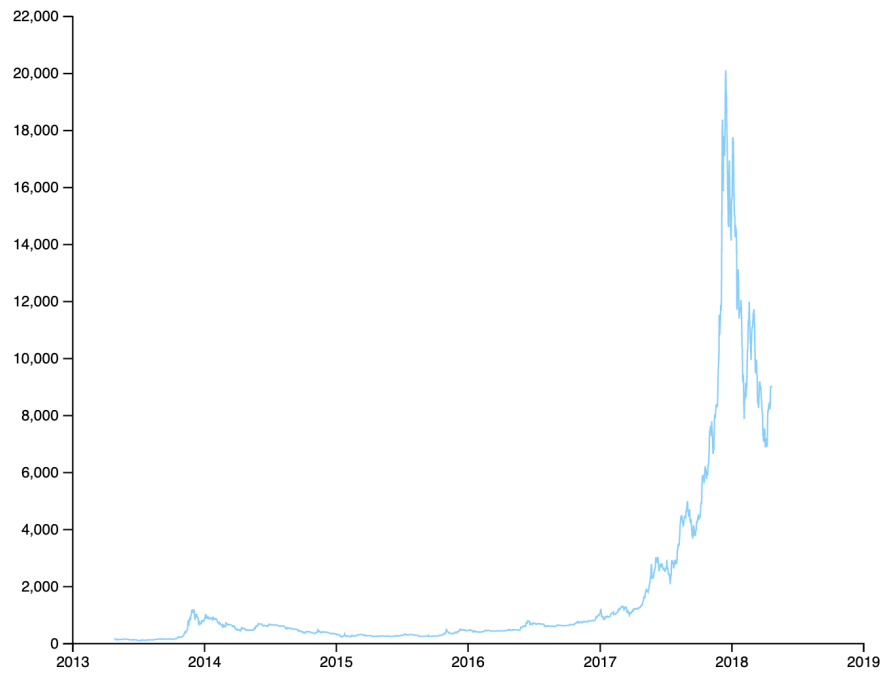
Per aggiungere gli assi, utilizziamo le funzioni d3 **.axisBottom()** e **.axisLeft()** che permettono di generare **automaticamente** gli assi con i vari **intervalli**.

Per mostrare questi assi è necessario **aggiungerli** al **DOM**, nello specifico nel gruppo che contiene il path generato.

Si aggiungono due ulteriori gruppi, uno per ogni asse.

```
function createAxisXD3(gContainer, xScale, hChart){ Show usages
  gContainer.append("g")
    .attr("transform", "translate(0, "+ hChart + ")")
    .call(d3.axisBottom(xScale))
}

function createAxisYD3(gContainer, yScale){ Show usages
  gContainer.append("g")
    .call(d3.axisLeft(yScale))
}
```



Esercizi

Svolgere l'esercizio 'linechart-tooltip' presente nella cartella associata a questa lezione.

L'esercizio richiede di creare un tooltip: seguendo i suggerimenti, usate le funzioni per fare in modo che quando il puntatore si muove nel grafico, vengano visualizzati gli assi e un box informativo che indica data e valore di quel preciso punto.

